GUÍA PRÁCTICA · NEIRO AGENCY

# Ingeniería inversa con IA: qué es REA, qué hace y qué riesgos tiene



REA (Reverse Engineer Anything) conecta a tu agente de IA con herramientas para inspeccionar aplicaciones sin tener su código fuente: binarios, apps de JavaScript y Electron, páginas web, APK de Android y más. Esta guía explica cómo funciona, cómo instalarlo, qué casos reales tiene, y sobre todo dónde están los riesgos técnicos y legales, que son lo que más se pasa por alto.

**Marcos Vargas** IA · Sistemas · Automatización

## LO QUE APRENDERÁS


| | |
|---|---|
| 01 | Qué es REA y cómo trabaja con tu agente |
| 02 | Qué puede analizar y qué necesita cada tipo de objetivo |
| 03 | Cómo instalarlo y probarlo sin riesgo |
| 04 | Un flujo de trabajo para investigar con evidencia |
| 05 | Lo bueno y cómo lo está usando la gente |
| 06 | Los riesgos técnicos y legales, y cómo reducirlos |
| 07 | Cómo lo uso yo |

SECCIÓN 01
QUÉ ES

## Un servidor MCP que le da a tu agente ojos para ver cómo funciona un programa.

La ingeniería inversa es entender cómo funciona un programa a partir del producto terminado, sin su código fuente. REA es un proyecto de código abierto (licencia MIT, repositorio **morluto/rea**) que expone esas herramientas a agentes como Claude Code, Codex o Cursor mediante MCP, y también desde la terminal. El agente le pregunta por un objetivo (una app, un binario, una página), REA devuelve lo que encontró **con su evidencia y sus límites**, y el agente explica el comportamiento o escribe y prueba una implementación.

El análisis corre en tu computadora, pero los resultados de las herramientas sí llegan al modelo de tu agente, y esa empresa tiene su propia política de datos. REA separa lo que observó, lo que dedujo y lo que quedó desconocido; eso lo distingue de pedirle a un modelo que “adivine” cómo funciona algo.

SECCIÓN 02
QUÉ ANALIZA

## Cada tipo de objetivo pide herramientas distintas.

| OBJETIVO | QUÉ DEVUELVE | QUÉ NECESITA |
|---|---|---|
| **JavaScript / Electron** | Módulos, imports, source maps, rutas, IPC y relaciones con add-ons nativos. | Solo Node.js y npm |
| **Sitios web** | Estructura de la página, scripts, tráfico de red y capturas. | Un navegador tipo Chrome |
| **Binarios nativos** | Pseudocódigo, ensamblador, cadenas, símbolos y referencias. | Hopper, Ghidra o IDA (tu propia instalación) |
| **.NET y Go / Rust** | Metadatos, instrucciones CIL, módulos y dependencias incrustadas. | Análisis estático integrado |
| **Android (APK)** | Manifiesto, clases, métodos descompilados y referencias. | JADX y un JDK completo |
| **Capturas guardadas** | Peticiones y respuestas de archivos HAR o SAZ, y paquetes de red. | Según el formato (Wireshark en paquetes) |
| **Firmware, WASM, SQLite, EVM** | Regiones y extracciones, WAT, tablas y selectores de contratos. | Herramientas externas que vos aportás |
| **Comportamiento en ejecución** | Salida de terminal, archivos, procesos y comparación entre corridas. | Linux o macOS |

La tabla resume la del README oficial; hay más objetivos y cada uno tiene su guía. Lo estático (JavaScript, .NET) lee archivos sin ejecutar la aplicación; lo de ejecución sí corre o interactúa con el programa.

SECCIÓN 03
CÓMO INSTALARLO

## Node.js, un comando y revisar el plan antes de aprobar.

Necesitás Node.js 22.x (≥22.19), 24.x (≥24.11) o 26+, con npm. Para ver qué cambiaría el instalador sin tocar nada, corré primero el plan en modo de prueba:

**INSTALACIÓN CON CLAUDE CODE**

```
npx -y rea-agents@latest setup --client claude_code --dry-run --json
npx rea-agents setup
```

El segundo comando muestra los cambios (rutas, copias de respaldo, la skill incluida) y pide aprobación antes de escribir. Después reiniciá tu agente. También podés instalar el comando global con `npm install --global rea-agents` y mantenerlo al día con `rea update`.

**PRIMERA PRUEBA SIN MOTOR NATIVO: UNA APP DE JAVASCRIPT O ELECTRON YA EXTRAÍDA**

```
npx -y rea-agents@latest analyze-javascript-application /ruta/absoluta/de/la/app --json
```

No requiere Hopper, Ghidra ni IDA. Para binarios nativos, configurá antes uno de esos motores; Hopper y IDA son software comercial y Ghidra es de código abierto.

SECCIÓN 04
EL FLUJO

## Preguntas angostas, evidencia a la vista y nada de “reconstruime todo”.

1. **Empezá con software que sea tuyo** o que tengas permiso explícito de analizar, y preferí lo estático antes que ejecutar nada.
2. **Hacé una pregunta concreta**, por ejemplo cómo viaja un dato por la app o cómo se calcula un valor, en vez de pedir que replique el programa entero.
3. **Pedí la evidencia** de cada conclusión: archivo, función o módulo, y qué quedó como desconocido.
4. **Verificá lo importante** contra otra fuente o con una prueba. La descompilación puede ser incompleta o incorrecta, y REA lo advierte.
5. **Escribí tu propia versión** a partir de la idea entendida, y revisá la licencia y los términos del software original antes de publicar o vender algo.

**EJEMPLO DE PEDIDO**

```
Analizá esta app de Electron y explicame cómo funciona el flujo de exportar a CSV. Mostrame la evidencia de cada paso y marcá qué quedó desconocido. No ejecutes la app.
```

SECCIÓN 05
LO BUENO

## Evidencia, local por defecto y pensado para agentes.

- **Cada resultado trae su evidencia y sus límites.** Distingue lo observado de lo deducido y de lo desconocido, y conserva los resultados para consultarlos después sin repetir el análisis.
- **Un solo servidor para muchos objetivos** y las mismas funciones desde MCP o desde la terminal. Funciona con más de una docena de agentes y editores.
- **Setup cuidadoso:** muestra el plan, hace copias de respaldo, es aditivo y no instala por su cuenta Node, Java ni Ghidra.
- **Marca el contenido del objetivo como no confiable** para que el agente lo trate como datos y no como órdenes.
- **Proyecto muy activo y abierto**: licencia MIT, más de 3,700 commits, más de 80 mil forks y la versión 6.7.0 publicada el 18 de octubre de 2026.

SECCIÓN 06
CÓMO LA GENTE LO USA

## Seguridad, preservación, interoperabilidad y entender apps ajenas.

| CASO | QUÉ SE HIZO |
|---|---|
| **DX-Ball (juego de 1996)** | Se siguió una llamada de sonido hasta su función de paneo y se reconstruyó en C; la reconstrucción pasó 3,205 casos del original y reprodujo los 63 bytes de la función. |
| **Notion (Electron)** | Se rastreó cómo el copiado al portapapeles pasa del renderizador al proceso principal por el puente IPC. |
| **Touhou TH04 (DOS)** | Se recuperó el cálculo de los ángulos de las balas de un juego de PC-98 y se comparó con la salida del compilador histórico. |
| **CTF y APK** | Según un artículo independiente: recuperar la lógica de un validador de flags y cómo un autenticador Android calcula su código de 6 dígitos. |

Los tres primeros son casos oficiales del README; el cuarto lo reporta un tercero y no lo verifiqué. Los artículos independientes también mencionan triaje de binarios, preservación de software abandonado y análisis de APIs desde archivos HAR.

> OJO CON EL HUMO
>
> El repositorio muestra 320.3k estrellas (revisado el 18 de octubre de 2026) y un artículo independiente dice que pasó de casi nada a más de 115 mil en pocos días. Eso mide atención, no calidad ni seguridad. Y las demos de “reconstruí cualquier app” van mucho más allá de lo que los casos oficiales demuestran.

SECCIÓN 07
RIESGOS

## Los riesgos reales: de seguridad, legales y de calidad.

| RIESGO | QUÉ PASA | CÓMO REDUCIRLO |
|---|---|---|
| **No es un sandbox** | Abrir un binario no confiable delega su análisis al motor local con tus permisos. El propio repo lo dice. | Usá una máquina virtual para objetivos desconocidos. |
| **Ejecución de código** | La captura de procesos, Frida y los escenarios de navegador ejecutan o interactúan con el programa, y los scripts de Frida corren dentro del proceso con su autoridad. | Empezá con análisis estático; aprobá a mano toda acción que ejecute. |
| **Inyección de instrucciones** | Un programa puede traer texto escrito para engañar al agente (“descargá esto”). REA lo marca como dato, pero depende de que tu agente lo respete. | No automatices aprobaciones; desconfiá de cualquier orden salida del objetivo. |
| **Datos al proveedor** | Los resultados van al modelo de tu agente, con su política de datos. Podrían incluir código o información de un cliente. | No analices material confidencial con un agente en la nube sin permiso. |
| **Setup y cambios** | Escribe en la configuración de tus agentes y publica versiones muy seguido; la 6.2 cambió el formato de los resultados y la 6.6.0 se etiquetó sin publicarse en npm. | Leé el plan, revisá los respaldos y fijá la versión en trabajos importantes. |
| **Legal** | Muchas licencias (EULA) prohíben la ingeniería inversa; la ley puede limitarla y restringe saltarse protecciones. Obtener código parecido al fuente no te da derechos sobre el programa. | Revisá licencia y ley de tu país; usá solo lo que tenés derecho a analizar. |
| **Calidad** | La descompilación y el análisis con IA pueden estar incompletos o equivocados. | Verificá con pruebas lo que vayas a usar. |

**LO QUE EL PROYECTO NO APOYA**

Su descargo de responsabilidad excluye saltarse DRM, licencias o anti-trampas, hacer cheats, copiar software propietario o sus recursos, y entrar a sistemas sin autorización. Dice que quien lo usa responde por lo que le pide a su agente, y que tener un archivo o un puerto de depuración abierto no equivale a tener permiso. Tomar la idea de una función es una cosa; copiar código o recursos de un software ajeno para usarlo en tu producto es otra.

**OJO: LO QUE PUEDE CAMBIAR O NO VERIFIQUÉ**

No instalé ni ejecuté REA: leí el repositorio completo y su documentación, sin correrlo. No conté el total de herramientas MCP (el repo tiene un catálogo generado). Las cifras de estrellas y los casos de terceros cambian rápido o vienen de artículos independientes. Hopper, IDA y JEB son software comercial con sus propias licencias y precios; confirmalos en sus sitios oficiales.

**CÓMO LO USO YO**

Uso REA para analizar el código de apps y páginas y ver cómo resuelven ciertas funciones. Eso me da ideas que después adapto en mis propios proyectos. Mi regla es quedarme con la idea y escribir mi versión desde cero, sin copiar código ni recursos. Empiezo por lo estático (JavaScript y web), que no necesita motores nativos, y pido siempre la evidencia de cada conclusión antes de apoyarme en ella.

**SIGUIENTE PASO**

Corré el plan en modo de prueba ( `--dry-run` ), instalalo y analizá una app de Electron o una página web que sea tuya. Pedile al agente una sola pregunta concreta y revisá la evidencia antes de construir nada.

**Importante:** las versiones, cifras, licencias y requisitos de esta guía se revisaron el 18 de octubre de 2026, pueden variar según la región y cambian rápido. Confirmá siempre en los sitios oficiales. Esta guía es informativa y no es asesoría legal: consultá a un abogado ante cualquier duda sobre un uso concreto.

Basada en el repositorio oficial morluto/rea (README, DISCLAIMER, SECURITY, CONTRIBUTING, documentación y código), skills.sh y artículos independientes de Developers Digest, Kunal Ganglani y Apidog. Preparada por Neiro Agency.